

IOAI 场

- 时间限制：5 分钟
- 存储空间：5 GB
- 提交文件大小： `solution.ipynb`、`custom_model.py` 合计 ≤ 1 MB (`solution.ipynb`, `custom_model.py` ≤ 1 MB together)
- 预训练模型：无——从头训练，评分时无法访问互联网
- Baseline 分数：31.2187

任务

Astana 市长希望用风格化的 IOAI 标志 (IOAI logo) 装饰这座城市。作为一名统计学家，他将一切事物——包括该标志 (logo) ——都视为空间函数 $F(x, y, \overline{W})$ ，其中 $x, y \in [0, 1]$ 表示 2D 平面上的坐标，而 $\overline{W}$ 是一组隐藏参数，用于定义字母颜色和角度等风格属性。

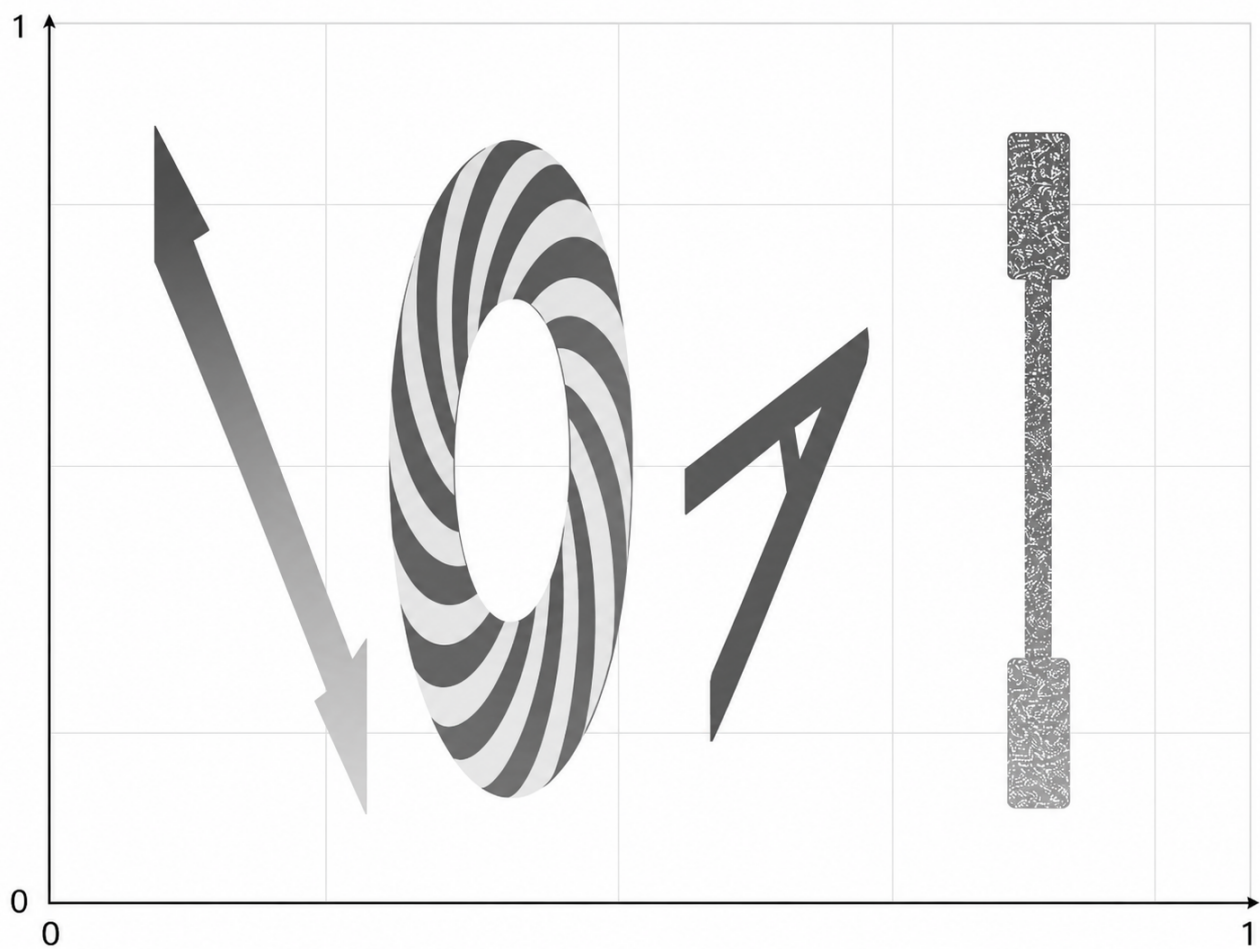
由于 F 过于复杂，无法表示为显式数学方程，因此你的任务是训练一个神经网络来近似它。该网络将针对任意坐标对 (x, y) 输出一个 **IOAI 场值** (IOAI field value)，从而生成该标志 (logo) 在整个平面上的完整热力图可视化结果。下面是具有某些特定隐藏参数 $\overline{W}$ 的 F 的热力图可视化示例。

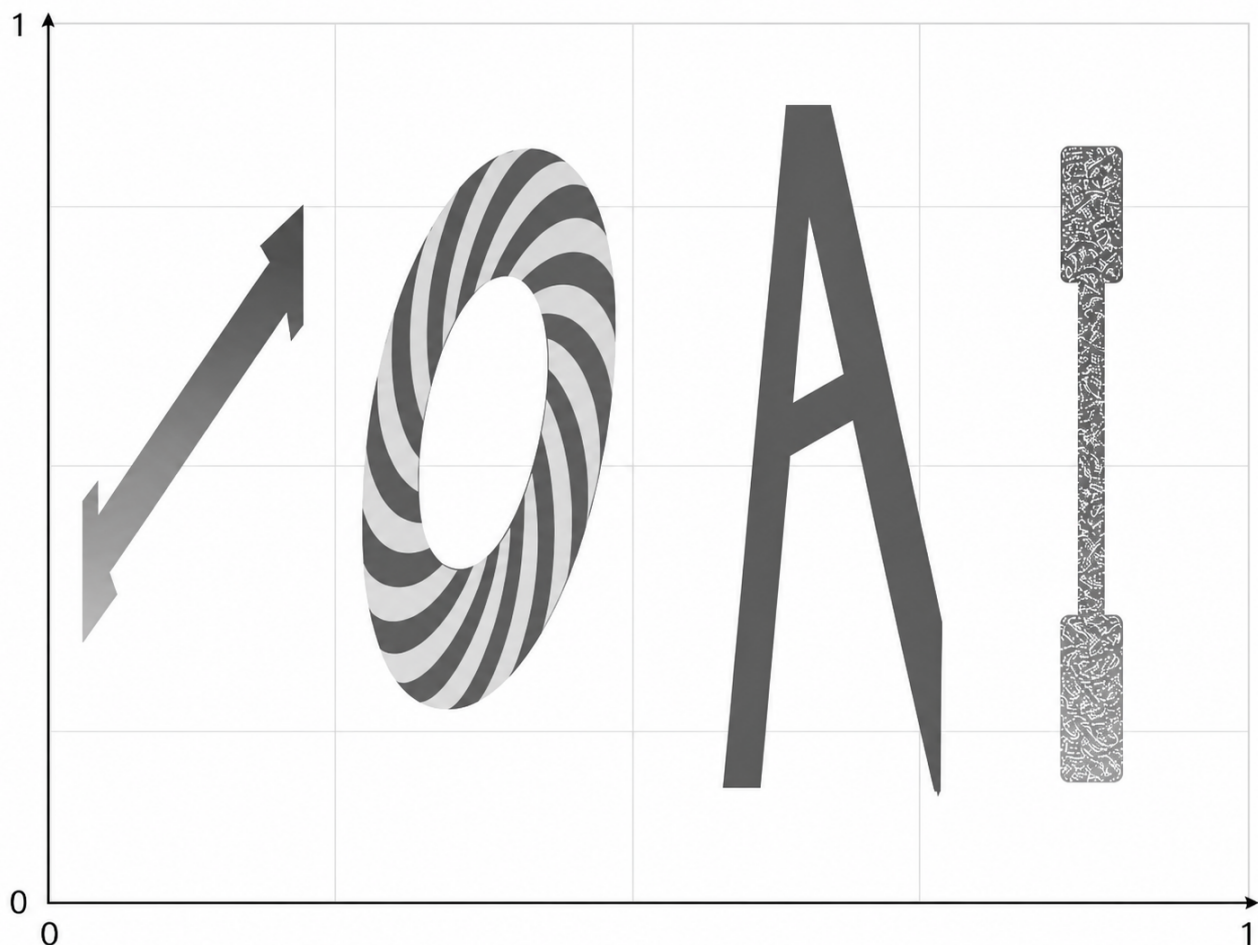


IOAI 场 (IOAI field) 由什么组成？四个字母和背景。

- 第一个 **I** 字母内部的值非常大 ($1e+10$ 及以上)，并具有线性梯度 (with a **linear gradient**)
- 字母 **O** 中的值呈现螺旋图案 (demonstrate spiral pattern)
- 字母 **A** 内部的值始终为 -1 (**always -1**)
- 最后一个 **I** 字母内部的值应为范围 $[-2026, 2026]$ 内的随机值，即使在同一点计算两次也是如此 (even if evaluated at the same point twice)
- 字母外部的值始终为零 (**always zero**)

该函数具有隐藏参数 $\overline{W}$ ，它会影响字母的缩放和倾斜，以及第一个 **I** 字母内部的取值范围。不过，字母不会相交 (**letters will not intersect**)。以下是 IOAI 场在不同 $\overline{W}$ 下外观的几个说明性示例：





给定内容：

本题不包含任何数据集。你得到的是生成器函数，该函数由位于 `data/train_config/field_config.json` 的 JSON 配置文件进行配置。

测试配置是隐藏的，但具有相似的性质。你的任务是使用任意多的数据对给定生成器进行拟合。你的“训练”和“测试”分布由同一个生成器生成——你只是不知道将在哪些点 (x_i, y_i) 上接受评测。

你的提交应包括：

- 保存为 `custom_model.py` 的待训练模型类。此模型应继承 `torch.nn.Module` 类，并且仅使用 `torch` 导入。它应包含 `CustomModel` 类，供 `solution.ipynb` notebook 使用。
- `solution.ipynb` notebook，它将生成 `model.pt` 权重

评分

对于每个区域，最低分为 0，最高分为 1。最终分数是在全部五个区域（四个字母各对应一个区域，另一个为背景区域）上的平均值，再乘以 100。此外还有一项**参数惩罚**：

如果你的模型参数多于 20260 个，分数将减半。

参数数量通过 `sum(p.numel() for p in model.parameters())` 测量。我们期望你的模型也能以随机模式运行，并将 `PyTorch nn.Dropout` 作为模型的一部分 (**We expect your model to operate in a stochastic mode as well with the PyTorch nn.Dropout being part of the model.**)。

对于标准区域

对于每个区域 R (第一个 `I` 字母、`O`、`A`、`Background`)，我们在 $N_R = 512$ 个测试点 (x_i, y_i) 上评估模型，其真实值为 v_i ，预测值为 $\hat{v}_i$ 。我们使用归一化平均绝对误差 (MAE) 作为主要指标。MAE 定义为：

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

归一化按如下方式进行：

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

其中 $s_R > 0$ 是缩放常数。

对于最后一个 `I` 字母区域

在该区域中，**评估期间会启用 dropout**。对于每个测试点 j ：

1. 我们运行模型 $K = 10$ 次，得到 $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$ 。
2. 如果任何输出超出范围 $[-2026, 2026]$ ，则 $\text{pointScore}(j) = 0$ 。
3. 否则，计算这 K 个输出的标准差 σ_j ，并将其转换为分数：

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

其中 $s_E > 0$ 是固定的缩放常数。

该区域的分数是区域内所有点所得分数的平均值：

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

其中 $N_E = K * N_R$ 。

简单来说，多样性越高，你在该区域获得的分数就越高。**你不能直接使用随机数，包括 `PyTorch rand*` 和 `_uniform` 函数；随机性应来自启用 dropout 后的推理过程。**

如何提交

1. 打开 `solution.ipynb` 并运行所有单元格。
2. 改进 `custom_model.py` 中的 `CustomModel` 模型

3. 确保最后一个单元格将模型保存到 `model.pt` 文件。
4. 在 JupyterLab 的 Git 选项卡中，暂存 `solution.ipynb` 和 `custom_model.py`，添加评论 (**comment and commit commit solution.ipynb and custom_model.py**) 并提交，然后推送。
5. 返回竞赛页面并点击 **Submit**。提交评论 (Submit comment) 应与上一步中的评论相同。